

DAY 1 · MODULE 4 · 40 MIN

MAF 101

Core primitives, in Python and C#

What MAF is

- Microsoft's SDK for building agents
- One vocabulary — Agent, chat client, tool, session, run
- Python (`agent_framework`) and C# (`Microsoft.Agents.AI`) with matching concepts
- First-class Foundry integration
- Streaming, memory, structured outputs, tools, MCP, multi-agent, eval — one place
- The successor to Semantic Kernel and AutoGen for new work

The primitives

Primitive	What it is
Chat client	Typed client that talks to a specific model service (e.g. FoundryChatClient)
Agent	Wraps a chat client with instructions and tools
Session	Carries conversation history across multiple agent.run() calls; create with agent.create_session(), pass as session=
Run	One turn (user → agent), non-streaming or streaming
Tool	A callable capability the model can invoke
Message	Individual user / assistant / tool messages in a session

The simplest possible Python agent

```
import asyncio
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

async def main():
    agent = Agent(
        client=FoundryChatClient(credential=AzureCliCredential()),
        name="HelloAgent",
        instructions="You are a friendly assistant. Keep answers brief.",
    )
    result = await agent.run("What is the capital of France?")
    print(result)

asyncio.run(main())
```

D E M O

~3 min

The 6-line agent

Live-type an empty scratch directory into a running MAF agent. Six lines from the previous slide — plus `uv init`, `uv sync`, and two `env vars` — get you a real answer from your Foundry-deployed model. Proves there's no ceremony.

Runbook: [demos/day1/module-4-demo-1-six-line-agent.md](#) · Scratch files: [demos/day1/module-4-demo-1-six-line-agent/](#)

The simplest possible C# agent

```
using Azure.AI.Projects;
using Azure.Identity;
using Microsoft.Agents.AI;

var endpoint = Environment.GetEnvironmentVariable("FOUNDRY_PROJECT_ENDPOINT")!;
var model = Environment.GetEnvironmentVariable("FOUNDRY_MODEL") ?? "gpt-5.6-luna";

AIAgent agent = new AIProjectClient(new Uri(endpoint), new DefaultAzureCredential())
    .AsAIAgent(model: model,
        name: "HelloAgent",
        instructions: "You are a friendly assistant. Keep answers brief.");

Console.WriteLine(await agent.RunAsync("What is the capital of France?"));
```

Non-streaming vs. streaming

```
# Python
result = await agent.run("Tell me a fun fact.")
print(result)

async for chunk in agent.run("Tell me a fun fact.", stream=True):
    if chunk.text:
        print(chunk.text, end="", flush=True)

// C#
Console.WriteLine(await agent.RunAsync("Tell me a fun fact.));

await foreach (var update in agent.RunStreamingAsync("Tell me a fun fact.))
{
    Console.Write(update);
}
```

Multi-turn conversations

AgentSession carries conversation state across runs. Create one with `agent.create_session()` and pass it to every `agent.run()` call.

```
session = agent.create_session()

r1 = await agent.run("My name is Alex.", session=session)
r2 = await agent.run("What's my name?", session=session)    # answer: Alex
```

You create a session once and pass it to each run. MAF maintains the conversation history in that session object for you. Without `session=`, each call is stateless. Day 3 covers explicit session management (persistence, compaction, replay).

Authentication

- Every MAF sample uses Azure identity, not API keys
- Dev on your laptop: `AzureCliCredential()` — signs in via `az login`
- CI / production: `DefaultAzureCredential()` or, preferably, an explicit `ManagedIdentityCredential`
- Never commit API keys or connection strings — enforced on Day 5

Packages you'll touch this week

Concern	Python package	C# package
Core	agent-framework	Microsoft.Agents.AI
Foundry client	agent-framework-foundry	Microsoft.Agents.AI.Foundry
Foundry hosting	agent-framework-foundry-hosting	(via Microsoft.Agents.AI.Foundry)
AI Search	agent-framework-azure-ai-search	Azure.Search.Documents
MCP hosting	agent-framework-hosting-mcp	ModelContextProtocol samples

Bridge to the next two modules

You've now seen the MAF primitives that let your process call the Foundry Responses API — Agent + FoundryChatClient (Python) or AIProjectClient.AsAIAgent (C#).

That's the self-hosted path — one of two hosting families for Agent Framework agents. The other family — Foundry-hosted — has two flavors: Prompt agents (configuration only) and Hosted agents (your code, containerized, run by Foundry). All are connected via FoundryAgent.

Module 5 gives us the mental model that ties everything together. Module 6 walks both hosting families in detail.

Takeaways

- MAF's primitives are small and stable: chat client, agent, session, run, tool, message.
- Python and C# APIs mirror each other closely.
- Auth is Azure identity end-to-end. No keys.
- The pattern you just saw — your code calling the Responses API — is the self-hosted family. Module 6 covers both families in detail.

Next: The five-layer agent stack we'll refer to through the rest of the workshop.